

Phase 1 - Week 2 - Lesson Day 1

Pandas Basics

Today's Objectives

Week 1 was all about NumPy - working with raw numeric arrays. This week introduces **Pandas**, the library you will use more than any other in this entire course. Pandas is built on top of NumPy, but adds labels, column names, and tools designed specifically for working with real-world tabular data - exactly like the spreadsheets and tables you already know from Excel and SQL.

- Understand the difference between a Series and a DataFrame
- Load data from CSV and Excel files into Pandas
- Use `.head()`, `.info()`, and `.describe()` to get a first look at any dataset

1. What Is Pandas? Series vs DataFrame

Definition: Pandas is a library for working with *labeled, tabular data* in Python - think spreadsheets or SQL tables, but with the full power of Python code available on top of them. It gives you two core structures.

Series - a single column of data, where every value has a label (called the index).

```
import pandas as pd    # 'pd' is the standard alias used everywhere

ages = pd.Series([25, 30, 35], name="age")
print(ages)
# 0    25
# 1    30
# 2    35
# Name: age, dtype: int64
```

Notice the numbers on the left (0, 1, 2) - that's the **index**. By default it's just row position, but it can be set to something meaningful, like dates or IDs.

DataFrame - a full table made of multiple Series (columns) sharing the same index.

```
df = pd.DataFrame({
    "name": ["Ali", "Halimo", "Amina"],
    "age": [25, 30, 28],
    "city": ["Mogadishu", "Hargeisa", "Kismayo"]
})
print(df)
```

Where it matters in data science: a DataFrame is essentially your Excel sheet or SQL table, loaded into memory as a Python object you can filter, transform, and analyze with code. Each column of a DataFrame is technically a Series.

2. Reading Data (CSV / Excel)

Definition: Pandas has built-in functions to load data directly from common file formats into a DataFrame, without you having to manually parse the file yourself.

```
df = pd.read_csv("data.csv")          # load a CSV file into a DataFrame
df = pd.read_excel("data.xlsx")        # load an Excel file into a DataFrame

# Since you already know Excel, this is the direct equivalent of opening a spreadsheet -
# except now the entire table lives inside a variable you can manipulate with code.
```

In Google Colab specifically, you'll usually either upload the file directly into the session (using the file upload button in the sidebar) or load it straight from a URL - we'll cover both properly in this week's exercises.

3. First Look at a Dataset

Definition: Before doing anything else with a new dataset, data scientists always run a handful of "first look" commands to understand its size, structure, and quality. Skipping this step is one of the most common beginner mistakes - it leads to bugs later that are much harder to trace back.

```
df.head()          # shows the first 5 rows
df.head(10)        # shows the first 10 rows instead
df.tail()          # shows the LAST 5 rows

df.info()          # column names, data types, and how many non-missing values each has

df.describe()      # summary statistics for numeric columns: count, mean, std, min, max, et
c.

df.shape           # (rows, columns) - similar to NumPy's .shape
df.columns         # a list of all column names
```

Command	What it tells you
.head()	What the data actually looks like (spot-check for obvious issues)
.info()	Column data types and missing value counts
.describe()	Summary statistics (mean, std, min, max, quartiles) for numeric columns
.shape	How many rows and columns you're working with

Connecting to what you already know: think of **.describe()** as the equivalent of Excel's summary statistics or a quick Power BI card visual, and **.info()** as checking your column data types before building a pivot table - except now it happens instantly with one line of code, on datasets far larger than Excel could comfortably handle.

Today's Exercise

Open the accompanying notebook file **day-1-pandas-basics.ipynb**. This exercise uses a dataset from Kaggle - pick any beginner-friendly dataset (for example, search Kaggle for "Titanic" or "Iris" - both are classic, small, and well-documented) and load it into Colab. If you're not sure how to download and upload it, ask in the lesson chat and we'll walk through it together.

Exercise Questions (8 total)

1. Load your dataset

Download a dataset from Kaggle, upload it into Colab, and load it into a DataFrame using `pd.read_csv()`.

2. First look

Run `.head()` to view the first 5 rows, then `.head(10)` to see 10 rows instead.

3. Last rows

Run `.tail()` to view the last 5 rows of the dataset.

4. Column info

Run `.info()` and, in a comment, write down how many columns have missing values.

5. Summary statistics

Run `.describe()` and identify (in a comment) which numeric column has the largest standard deviation.

6. Shape check

Print `df.shape` and state in a comment how many rows and how many columns your dataset has.

7. Column names

Print `df.columns` and list which columns look numeric vs which look categorical (text-based), based on their names.

8. Build your own Series and DataFrame

Separately from your dataset, manually create a Series of 5 numbers and a DataFrame with at least 2 columns and 3 rows of your own made-up data.

How to Approach It

Pick a dataset that genuinely interests you - you'll be working with it across the rest of this week (cleaning, filtering, grouping), so something you find mildly interesting makes the repetition far less tedious. Titanic (predicting survival) and Iris (flower measurements) are the two most beginner-friendly options if you're not sure where to start.

Once you and Halimo have completed the exercises, bring your findings back to the lesson chat and we'll review them before moving to Lesson Day 2 (Data Selection & Filtering).